Same Signal, Opposite Meaning: Why Adaptive Compute Fails Across Environments

Anonymous

Abstract

Test-time compute improves decision quality for LLM agents, but deciding when it is beneficial remains unresolved. Existing gating methods rely on uncertainty signals, implicitly assuming a fixed relationship between signal value and compute benefit. We show that this assumption is incorrect. Across eight diverse agent environments and three model backbones, we find that the same signal value that predicts rollout benefit in one environment predicts rollout harm in another, indicating a reversal in the signal–utility relationship. We trace this to a conflation of two uncertainty sources, information poverty and decision difficulty, and formalize it via a two-source model grounded in Simpson's paradox, where subgroup-level trends reverse upon aggregation. We show that any gating strategy with a fixed signal–utility direction must underperform never-gating in at least one such environment, making direction discovery unavoidable for cross-environment robustness. Motivated by this result, we propose EAAG (Environment-Aware Adaptive Gating), which learns environment-specific signal–utility relationships through direction discovery. Across eight environments, EAAG outperforms all fixed-direction baselines with zero additional per-step overhead, achieving adaptive gating behavior solely by learning signal direction.

1 Introduction

Large language models (LLMs) have rapidly advanced across a wide range of tasks, from question answering and code generation to web navigation and embodied decision making (Yao et al., 2023b; Shinn et al., 2023; Zhou et al., 2024). Yet, many tasks remain beyond the reach of a single forward pass: complex reasoning, multi-step planning, and open-ended exploration often require the model to evaluate alternatives, verify intermediate results, or search over candidate actions (Yao et al., 2023a; Hao et al., 2023; Snell et al., 2024). This has motivated test-time compute, where additional computation at inference time improves decision quality. Across multiple benchmarks, test-time optimization allows smaller models to match or exceed the performance of larger ones at lower total cost (Snell et al., 2024; Wang et al., 2026).

While effective, test-time compute is expensive, incurring substantial per-step overhead (e.g., requiring multiple rollouts or candidate evaluations at each decision point) (Yao et al., 2023a; Zhou et al., 2024). As agents are deployed across diverse settings, applying this overhead uniformly becomes wasteful when only a fraction of steps benefit. This has motivated adaptive gating: using uncertainty signals to decide when extra compute is worthwhile. Prior work explores diverse mechanisms, including entropy thresholding (Lee et al., 2025; Huang et al., 2026; Jin et al., 2026), calibrated confidence (Muennighoff et al., 2025; Han et al., 2025), vote disagreement (Lee et al., 2026), and learned difficulty estimators based on hidden states or reinforcement learning (Liu et al., 2025; Zhu et al., 2025; Zhang et al., 2025; Fang et al., 2025). Across these mechanisms, adaptive gating consistently reduces cost while preserving or improving task performance within each method's evaluation setting.

However, this consensus rests on a narrow empirical base. The fixed-direction assumption holds well in the settings where these methods have been developed and evaluated, predominantly homogeneous reasoning benchmarks where signal direction is consistent by

construction. Whether it transfers to heterogeneous agent environments is an open question prior work has not directly addressed, and we show that it does not: the relationship between uncertainty and compute value can reverse direction across environments, and adaptive gating can actively underperform never invoking the optimizer. This raises a central question: what determines the relationship between uncertainty signals and the value of computation, and can this relationship be learned in a way that generalizes across environments?

We answer this through systematic measurement across eight agent environments spanning QA (Yang et al., 2018), code generation (Hendrycks et al., 2021), web navigation (Yao et al., 2022), fact verification (Thorne et al., 2018), and text games (Jansen and Côté, 2023), using three model backbones. The signal–utility relationship reverses direction across environments: the same signal that predicts rollout benefit in code generation predicts rollout harm in fact verification, and can further reverse across model backbones within the same environment. Moreover, the most informative signal itself varies across environments, with structural features dominating in some settings while entropy carries little or no information in others.

We trace this reversal to a conflation of two distinct sources of uncertainty. Information poverty (Type I) arises when the agent lacks sufficient information; rollouts from the same model cannot compensate, and high uncertainty predicts negative value of computation. In contrast, decision difficulty (Type D) arises when multiple viable options exist; additional computation helps explore alternatives, and high uncertainty predicts positive value. Code generation often involves multiple valid implementations (Type D), while fact verification may lack sufficient evidence altogether (Type I). Because environments differ in their mixture of these two state types, the same uncertainty signal carries opposite meanings depending on which source dominates, a phenomenon closely related to Simpson's paradox (Simpson, 1951; Wang et al., 2023). The consequence is sharp: no fixed-direction gating strategy can avoid net-harmful gating across environments with opposing signal directions (Proposition 1), and once the direction is misaligned, improved calibration can worsen performance by more precisely selecting harmful states.

Motivated by this analysis, we propose EAAG (Environment-Aware Adaptive Gating), a deliberately minimal method that learns environment-specific signal–utility relationships from interaction data. Since the primary challenge is identifying signal direction rather than modeling capacity, EAAG's contribution is not a new gate architecture but a pipeline that reduces the adaptive-gating problem to a form where a standard sparse linear classifier suffices, adding zero per-step cost at deployment. Across eight environments and three LLM backbones, EAAG outperforms all fixed-direction baselines.

Taken together, these results point to a more fundamental observation: uncertainty signals like token entropy are not sufficient statistics for the value of computation, because their meaning depends on a latent state property that the signal alone cannot identify. Direction discovery is the minimal mechanism that closes this gap.

Our main contributions are as follows:

- A structural limitation of uncertainty-based gating. We show that uncertainty signals are not sufficient statistics for the value of computation: their meaning depends on a latent state type, and as a consequence no fixed-direction gate can be universally correct.
- Empirical finding. We demonstrate systematic reversal of signal–utility direction across environments and model backbones, and show that the most informative signals vary across settings.
- Adaptive gating method. We propose EAAG, which learns signal direction per environment instead of assuming it, outperforming all fixed-direction baselines across eight environments with zero deployment overhead.

2 Related Work

Prior work on adaptive test-time compute spans diverse mechanisms but rests on a shared assumption: that uncertainty signals

have a fixed, environment-independent relationship with compute value.

Gating on uncertainty signals. One line of work allocates extra computation based on token confidence, entropy, or disagreement, under the assumption that higher uncertainty indicates greater need for compute. In the reasoning setting, signal-based methods extend chain-of-thought generation when token confidence or entropy exceeds a threshold (Lee et al., 2025; Huang et al., 2026; Jin et al., 2026; Sharma and Chopra, 2025; Muennighoff et al., 2025; Han et al., 2025). In the agent setting, vote-based methods (Lee et al., 2026) sample multiple candidate actions and invoke an arbiter when they disagree, while verbalized uncertainty methods (Zhang et al., 2026) prompt the agent to reflect when its self-reported confidence is low.

Gating on learned difficulty. A second line learns to predict task difficulty and allocate compute accordingly. Hidden-state probes (Liu et al., 2025; Zhu et al., 2025) estimate difficulty from internal representations, and RL-based methods (Zhang et al., 2025; Fang et al., 2025; Aggarwal and Welleck, 2025; Paglieri et al., 2025) learn think/no-think policies end-to-end. Though these methods do not commit to a specific signal, their gating policies effectively behave as if higher difficulty warrants more compute, sharing the same fixed, environment-independent direction as the first family. This assumption holds well in the settings where these methods have been developed and evaluated, predominantly homogeneous reasoning benchmarks such as MATH, GSM8K, and GPQA, where signal direction is consistent by construction. Whether it transfers to heterogeneous agent environments is an open question that prior work has not directly addressed; we show it does not (Proposition 1). A per-method comparison is in Appendix A.

Orthogonal: search-based optimizers. Search methods such as Tree of Thoughts (Yao et al., 2023a), LATS (Zhou et al., 2024), RAP (Hao et al., 2023), and FLARE (Wang et al., 2026) improve what the optimizer does through tree search, value propagation, and lookahead evaluation. These are orthogonal to our work: we control when to invoke the optimizer, and our gate composes with any such procedure.

Table 1: Signal–utility correlations across 8 environments and 3 backbones. ρ: Spearman correlation of token entropy with optimizer utility U. *: significant at $p<0.05$. Direction varies across both environments and model backbones.

Environment	Qwen3	Phi-3.5	Llama-3.1	Cons.
HotpotQA	-0.327^*	$+0.184^*$	-0.346^*	✗
APPS Intro	$+0.144^*$	-0.129^*	-0.242^*	✗
WebShop	$+0.133^*$	$+0.335^*$	$+0.287^*$	✓
FEVER	-0.119^*	-0.156^*	$+0.428^*$	✗
APPS Intv.	$+0.317^*$	-0.024	-0.249^*	✗
TWExpress	-0.290^*	$+0.000$	$+0.000$	—
CRUXEval	-0.048	-0.065	-0.045	—
Plancraft	-0.016	$+0.000$	-0.176^*	—

Metareasoning and uncertainty semantics. Rational metareasoning (Russell and Wefald, 1991; Sabbata et al., 2024) formalizes the value of computation (VOC), classically assuming VOC ≥ 0 since an agent can disregard unfavorable computations. We show that this assumption fails in the agent setting, where the same model both proposes and evaluates actions: under this evaluator–executor identity, VOC can be negative, making direction discovery necessary. (Tao et al., 2025) and (Heo et al., 2025) find that calibration quality varies across task types, and (Abbasi Yadkori et al., 2024) decouple epistemic and aleatoric uncertainty—providing converging evidence that uncertainty semantics are domain-dependent, though they stop short of documenting direction reversal. Our two-source model formalizes this domain dependence (§3.2), and our reversal results make it actionable.

3 Why Fixed-Direction Gating Fails

We test the fixed-direction assumption by measuring signal–utility correlations across diverse environments. If the assumption holds, correlations should be consistent; if it fails, the failure mode will guide method design.

We study LLM-based agents in multi-step interactive environments. At each step t, the agent observes state s_t, selects action a_t, and transitions to s_{t+1}. An environment-specific test-time optimizer T (e.g., rollout evaluation, K-variant sampling) can be invoked at any step to potentially improve action quality. We

define the optimizer utility as:

$$U(T, s) = \mathbb{E}[R(\tau) \mid a{=}T(s)] - \mathbb{E}[R(\tau) \mid a{=}\pi(s)] \quad (1)$$

where $R(\tau)$ is the episode return. $U > 0$ indicates the optimizer improves the outcome; $U \leq 0$ indicates it is wasteful or harmful. The adaptive gating problem is to learn a gate $g : \mathcal{S} \to \{0, 1\}$ that triggers T only when $U > 0$.

3.1 Signal–Utility Direction Reverses Across Environments and Backbones

The signal–utility direction is not fixed. Table 1 reports Spearman correlations between token entropy and optimizer utility across 8 environments and 3 model backbones. Two patterns stand out. First, even within a single backbone, the direction varies across environments: on Qwen3, entropy is negatively correlated with utility in fact-verification environments (FEVER, TWExpress) but positively in code generation (APPS Interview), with near-zero correlation in mixed environments. Second, the same signal in the same environment can reverse across backbones: in 4 out of 5 environments with multiple significant correlations, models disagree on the sign. Most strikingly, FEVER flips from $\rho{=}-0.156$ on Phi-3.5 to $\rho{=}+0.428$ on Llama-3.1. Structural signals such as step_count are more stable across backbones (Appendix C), while entropy is unpredictable without per-environment, per-model measurement. Crucially, this direction mismatch is not benign: as Section 5.2 will show, gates that assume the wrong direction systematically fall below the no-trigger baseline, making them strictly worse than doing nothing in mismatched environments.

Signal identity also varies across environments. Beyond direction, the most informative signal itself differs from environment to environment: step_count dominates in most environments, but WebShop instead relies on num_available_actions and task-specific features (Appendix D.1). Moreover, even the best single signal is near-chance: per-environment AUC analysis shows single token entropy at AUC≈0.50, while multi-signal logistic gates reach AUC≈0.83 and hidden-state probes reach AUC≈0.90 (Appendix D.3). Predicting rollout value requires multiple signals selected per environment, not a single universal scalar.

3.2 Why Does Direction Reverse? A Two-Source Model

The findings above establish that fixed-direction gating fails; we now ask why. We trace direction reversal to a minimal mechanism that jointly accounts for the observed patterns: the conflation of two distinct sources of uncertainty.

Intuition. Consider a web-shopping agent choosing between products. When it hesitates (high entropy) because multiple items match the query, exploring alternatives via rollouts is valuable, since the agent has adequate information but faces a genuine choice. Now consider a fact-verification agent that hesitates because the retrieved evidence is ambiguous. Here, high entropy reflects missing information, not optionality: rollouts from the same model with the same evidence will not resolve the ambiguity. In the first case, entropy positively predicts rollout utility; in the second, it predicts rollout futility. The same numerical signal carries opposite meaning because the source of uncertainty differs.

Model. We model direction reversal as arising from two sources of uncertainty at each decision step:

- Type I (information poverty): The agent lacks sufficient information. Rollouts from the same model cannot compensate, so utility is negatively related to entropy: $U_I(s) \sim -\alpha H(s) + \varepsilon_I$, $\alpha > 0$.
- Type D (decision difficulty): The agent has adequate information but faces multiple viable paths. Rollouts exploit this diversity, so utility is positively related to entropy: $U_D(s) \sim +\beta H(s) + \varepsilon_D$, $\beta > 0$.

Let $p_I(\mathcal{E})$ denote the fraction of Type I states in environment $\mathcal{E}$. The marginal correlation becomes

$$\rho(\mathcal{E}) \;\approx\; \beta - (\alpha + \beta)\, p_I(\mathcal{E}), \quad (2)$$

which reverses sign at the critical threshold $p_I^* = \beta/(\alpha + \beta)$ (full derivation in Appendix B.4). Environments with $p_I > p_I^*$ exhibit negative ρ (Type I dominated); those with $p_I < p_I^*$ exhibit positive ρ (Type D dominated); near p_I^*, the two effects cancel and en-

tropy carries negligible signal. This prediction is consistent with the observed Qwen3 pattern in Table 1: search-based QA (FEVER, HotpotQA, TWExpress) shows negative ρ, code generation (APPS Interview) shows positive ρ, and mixed environments cluster near zero. A full per-environment mapping is provided in Appendix B.5. Multi-backbone results (Appendix C) further show that the model backbone modulates the effective p_I, making direction unpredictable from environment structure alone. We treat this two-source decomposition as a minimal falsifiable model rather than a post-hoc explanation: in Section 5.4 we derive three predictions from it (temporal dynamics, cross-environment consistency, and signal identity alignment) and verify each on held-out data.

Theoretical grounding. This direction reversal is a special case of Simpson's paradox (Simpson, 1951; Pearl, 2022): aggregating subpopulations with opposing within-group trends reverses the aggregate correlation. Our decomposition adapts the epistemic/aleatoric distinction (Der Kiureghian and Ditlevsen, 2009) to the meta-decision setting: Type I states exhibit epistemic-like uncertainty (information deficit), while Type D states exhibit aleatoric-like diversity (multiple viable paths).

3.3 Necessity of Direction Discovery

Proposition 1 (Necessity of Direction Discovery). Suppose there exist two environments $\mathcal{E}_1, \mathcal{E}_2$ with opposite true directions: $d^*(\mathcal{E}_1) = +1$ and $d^*(\mathcal{E}_2) = -1$. Then no fixed-direction gate g_d can simultaneously satisfy $\mathrm{SR}(g_d, \mathcal{E}_1) \geq \mathrm{SR}(\mathrm{base}, \mathcal{E}_1)$ and $\mathrm{SR}(g_d, \mathcal{E}_2) \geq \mathrm{SR}(\mathrm{base}, \mathcal{E}_2)$.

The intuition is direct: a gate with $d = +1$ triggers on high-signal states, which in a Type I environment correspond exactly to states where rollouts harm the agent; the symmetric argument holds for $d = -1$ in Type D environments. Since real-world deployments span both environment types, no fixed-direction strategy can deliver non-negative value of computation universally—direction discovery is a necessary precondition for cross-environment robustness, not a refinement of it. The full proof is in Appendix B.1, and as Section 5.2 will show, every fixed-direction baseline empirically underperforms the no-trigger baseline in at least 2 of 8 environments.

4 Method: Environment-Aware Adaptive Gating

Section 3 imposes three requirements on any gating method that aims for cross-environment robustness. (i) Signal direction must be learned per environment, since Proposition 1 rules out any fixed choice. (ii) The feature set must be multi-signal, since no single signal is reliably informative across environments (§3.1). (iii) Exploration must be signal-agnostic, since conditioning on a signal whose semantics are unknown would bias the collected data in a direction that depends on the answer.

EAAG is a deliberately minimal instantiation of these requirements. Its contribution is not a new gate architecture but a pipeline that reduces the adaptive-gating problem to a form where a standard sparse linear model suffices: randomized exploration (§4.1) handles the unknown prior direction, an optional LLM feature layer (§4.2) proposes task-specific signals when universal ones are inadequate, and a sparse linear gate (§4.3) recovers per-environment direction from the resulting dataset. Algorithm 1 summarizes the full pipeline and its connection to requirements (i)–(iii).

Why a minimal gate suffices. The real difficulty identified in Section 3 is identifying signal direction per environment, not classification capacity. Once exploration and feature selection have produced unbiased (s, U) data and a usable feature pool, the remaining task is well-posed for a standard sparse linear model. On HotpotQA (Table 5), a logistic gate, an MLP, and a 2560-dimensional hidden-state probe all reach ∼95% SR with the correct direction; under a reversed direction, the MLP falls below the no-trigger baseline because a more expressive gate more precisely targets harmful states. In the environments we study, the marginal return on gate complexity is small compared to the return on getting direction right, so we invest complexity in exploration and feature selection and keep the deployed gate minimal.

4.1 Explore: Signal-Agnostic Data Collection

Exploration collects the (s, U) pairs that will be used to fit the gate. The central constraint is requirement (iii): the collection policy must not condition on any signal whose semantics we are trying to learn. Any signal-dependent alternative, such as uncertainty sampling, would condition on a signal whose relationship to U is exactly what §3.1 showed to be unknown, biasing $\mathcal{D}$ in a direction that depends on the quantity being estimated. Warm-starting from prior-method data inherits the same circularity.

EAAG therefore uses the simplest policy consistent with this constraint. At each step t in episode i, a Bernoulli coin flip determines the gating decision:

$$z_t^{(i)} \sim \text{Bernoulli}(\varepsilon_{\text{explore}}), \quad \varepsilon_{\text{explore}} = 0.5, \tag{3}$$

where z_t=1 triggers the optimizer T and z_t=0 uses the base action. Over $N_{\text{explore}} = 50$ episodes, this yields an unbiased dataset

$$\mathcal{D} = \left\{ \left(\phi(s_t^{(i)}),\ U_t^{(i)} \right) : z_t^{(i)} = 1 \right\}_{t,i}, \tag{4}$$

where $U_t^{(i)} \in \{0, 1\}$ indicates whether the optimizer improved the outcome at step t, and $\phi(s)$ is the feature vector described in §4.2.

4.2 Reason: LLM Feature Layer

The next step builds the feature pool $\phi_{\text{cand}}(s)$ that the gate will be fit over. Requirement (ii) demands that this pool be rich enough to include each environment's relevant signals, but not so environment-specific that deployment requires manual engineering per task. EAAG's default is a fixed set of universal features: step_count, token_entropy, state_length, action_entropy, and others. These are available in every environment without task-specific engineering. In the environments we study, this pool alone is sufficient to recover a gate that outperforms all fixed-direction baselines (§5.3).

For environments where universal features may be inadequate, such as WebShop, whose gating benefits from task-specific signals like num_available_actions, EAAG offers an optional automation layer: an LLM is given a summary of $\mathcal{D}$ and proposes task-specific feature hypotheses $\phi_{\text{LLM}}(s)$, which join the universal pool to form

$$\phi_{\text{cand}}(s) = \phi_{\text{univ}}(s) \ \cup \ \phi_{\text{LLM}}(s). \tag{5}$$

The ablation in §5.3 shows its gains concentrate in the few settings where universal features miss the relevant structure, so the LLM is best viewed as an automation convenience for zero-shot deployment rather than a core component the method depends on.

4.3 Learn: Sparse Linear Gate

Given $\mathcal{D}$ and ϕ_{cand}, EAAG fits an ℓ_1-regularized logistic regression:

$$\mathbf{w}^*, b^* = \arg\min_{\mathbf{w}, b} \sum_{(\phi, U) \in \mathcal{D}} \ell\big(\sigma(\mathbf{w}^\top \phi + b),\, U\big) + \lambda \|\mathbf{w}\|_1 \tag{6}$$

where ℓ is the binary cross-entropy loss and λ controls sparsity over ϕ_{cand} so that uninformative features drop out. The deployed gate is

$$g(s) = \mathbf{1}\Big[\sigma({\mathbf{w}^*}^\top \phi(s) + b^*) > \tau\Big], \tag{7}$$

with threshold τ set by cross-validation on $\mathcal{D}$. Gate fitting completes in under a second, and deployment adds zero per-step overhead beyond feature computation. Implementation details are in Appendix E.2.

Signed weights as a diagnostic. The ℓ_1 penalty in Eq. 6 simultaneously performs feature selection and direction discovery. The sign of each fitted weight provides an interpretable per-environment estimate of the Two-Source mixture of §3.2: $w_i > 0$ indicates the environment sits on the Type D side of the reversal threshold p_I^* for signal i (high signal $\rightarrow$ trigger), $w_i < 0$ the Type I side (high signal $\rightarrow$ don't trigger), and $w_i \approx 0$ an uninformative signal. This is a diagnostic reading of the fitted model, not a novel modeling choice; any signed linear classifier would expose the same information.

Online adaptation. For drifting environments, EAAG can optionally refit during deployment using ε-greedy exploration (ε : $0.1 \rightarrow 0$) and periodic retraining. This mode is not free: ε-greedy overrides ignore the current gate, so in Type I environments a fraction of overridden steps will trigger harmful rollouts. For stationary environments, which are the setting of our main experiments, we recom-

Algorithm 1 EAAG: Environment-Aware Adaptive Gating

Require: Environment E, agent M, optimizer T, N_{explore} episodes, $\varepsilon_{\text{explore}}$=0.5
Ensure: Fitted gate $g(s)$

Phase 1: Explore // Req (iii)
$\mathcal{D} \leftarrow \emptyset$
for episode $i = 1, \ldots, N_{\text{explore}}$ do
  for step t in episode i do
    $z_t \sim \text{Bernoulli}(\varepsilon_{\text{explore}})$
    Execute T if z_t=1, else base action
    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\phi(s_t),\, U_t,\, z_t)\}$
  end for
end for

Phase 2: Reason (optional) // Req (ii)
$\phi_{\text{univ}} \leftarrow [\text{step_count}, \text{entropy}, \ldots]$
$\phi_{\text{LLM}} \leftarrow \text{LLM_Propose}(\text{summary}(\mathcal{D}))$
$\phi_{\text{cand}} \leftarrow \phi_{\text{univ}} \cup \phi_{\text{LLM}}$

Phase 3: Learn // Req (i)
$\mathbf{w}^*, b^* \leftarrow \arg\min_{\mathbf{w},b} \sum \ell(\sigma(\mathbf{w}^\top \phi + b), U) + \lambda \|\mathbf{w}\|_1$
$g(s) \leftarrow \mathbf{1}[\sigma(\mathbf{w}^{*\top} \phi(s) + b^*) > \tau]$

return $g(s)$ // Deploy: zero per-step overhead

mend skipping online adaptation and relying on the offline-fitted gate.

5 Experiments

5.1 Experimental Setup

We evaluate EAAG along three axes: does environment-aware gating Pareto-dominate fixed-direction baselines (§5.2); which components drive the gains (§5.3); and do the Two-Source Model's predictions hold on held-out data (§5.4).

Environments. We evaluate on 8 environments spanning QA, code generation, web navigation, fact verification, text games, and crafting planning (Table 2): HotpotQA, APPS Intro, WebShop, FEVER, TWExpress, Plancraft, APPS Interview, and CRUXEval. These environments are selected to cover the full range of the Two-Source Model, from pure Type I (FEVER) through mixed ($\approx p_I^*$, APPS Intro) to Type D (APPS Interview), including extreme rollout properties (TWExpress: rollout-safe; Plancraft: rollout-harmful).

Baselines. All methods share the same agent and optimizer T; only the gating decision differs. Bounds: base_only (never trigger), always_trigger, oracle. Fixed-direction baselines: CaTS, SEAG, CoRefine, CATTS, AUQ, s1_budget—all assuming high uncertainty warrants compute. Ablation: BSW (direction intentionally reversed). Full specifications in Appendix E.

Table 2: Environment setup. Base/Always: SR without/with optimizer. Δ: rollout headroom.

Environment	Base SR	Always SR	Δ	Optimizer T
HotpotQA	49.0%	97.0%	+48.0	Per-action eval
APPS Intro	58.5%	64.5%	+6.0	K-variant sampling
WebShop	7.2%	43.0%	+35.8	LLM-Propose-K
FEVER	37.0%	99.8%	+62.8	Per-action eval
TWExpress	67.5%	99.3%	+31.8	Per-action eval
Plancraft	29.8%	22.8%	−7.0	Per-action eval
APPS Interview	60.5%	79.5%	+19.0	K-variant sampling
CRUXEval	85.0%	99.5%	+14.5	K-variant sampling

Metrics. SR (success rate, ↑) and Cost (rollouts per episode during deployment, ↓). A method Pareto-dominates another if SR $\geq$ and Cost $\leq$ with at least one strict inequality. Cost measures only the deployment rollout budget; calibration or exploration overhead is excluded for all methods, ensuring a fair comparison of runtime efficiency.

5.2 Main Results

EAAG Pareto-dominates fixed-direction baselines. Table 3 reports SR and cost across 8 environments (summary in Table 4). EAAG matches or exceeds every fixed-direction baseline in SR while using fewer rollouts; the two exceptions occur in ceiling-effect environments where EAAG already approaches the oracle. No calibration data is required (statistical details in Appendix F). Beyond aggregate numbers, EAAG exhibits qualitatively different gating behavior: its trigger rate, direction, and signal weights all adapt to the environment, as we analyze below.

Cross-environment patterns. Figure 1 shows that the results align with the Two-Source Model along three axes. The gap between EAAG and baselines is largest where direction mismatch is most severe: in Type I environments, all "high uncertainty → trigger" baselines fall below the no-trigger baseline because the assumed direction is wrong—no amount of calibration rescues them. Conversely, EAAG's advantage is smallest in high-headroom environments where most methods approach the oracle regardless of direction. Figure 2 reveals the mechanism: in Type I environments, utility decreases with entropy,

Table 3: Main results across 8 environments (Qwen3-4B backbone). SR (%) and Cost (rollouts/episode). Bold: best SR per environment (excluding oracle). †: methods requiring Phase 1 calibration data. Δ: SR improvement over base_only.

	HotpotQA		APPS		WebShop		FEVER		TWExpress		Plancraft		APPS Intv		CRUXEval	
Method	SR	Cost	SR	Cost	SR	Cost	SR	Cost	SR	Cost	SR	Cost	SR	Cost	SR	Cost
Bounds																
base_only	49.0	0.00	58.5	0.00	7.2	0.00	37.0	0.00	67.5	0.00	29.8	0.00	60.5	0.00	85.0	0.00
always_trigger	97.0	1.80	64.5	2.58	43.0	5.63	99.8	1.46	99.3	3.45	22.8	6.99	79.5	2.19	99.5	1.91
Fixed-direction baselines																
CaTS†	93.2	1.06	59.0	0.04	30.5	3.05	50.2	4.71	96.7	1.97	22.3	4.39	66.2	0.54	95.0	0.73
SEAG†	67.5	0.80	58.5	0.01	28.0	2.28	49.3	3.12	97.3	2.30	24.8	2.16	66.0	0.64	85.8	0.73
CoRefine†	68.2	0.79	58.5	0.01	27.5	2.21	49.8	3.12	97.5	2.26	22.8	2.06	67.5	0.62	85.8	0.73
CATTS	68.3	1.07	58.5	0.03	16.0	0.19	34.2	0.06	97.5	2.26	25.0	2.14	60.8	0.02	81.3	0.04
AUQ	97.0	1.69	61.3	1.73	35.7	5.33	40.7	1.17	95.5	1.24	24.2	6.78	64.7	1.08	99.0	1.75
s1_budget	97.0	1.04	63.7	1.00	7.8	1.00	46.2	1.58	95.0	1.09	18.3	1.68	69.0	1.00	86.5	1.00
Ablation																
BSW	58.2	—	58.8	0.00	20.6	—	63.0	4.30	99.0	3.23	25.5	5.87	79.5	1.39	87.5	1.00
Ours																
EAAG	95.2	1.34	66.0	1.20	43.8	2.29	49.8	2.99	99.0	2.84	23.3	3.69	73.0	1.35	98.5	1.24

Table 4: Win/loss summary: EAAG vs. each fixed-direction baseline across environments. A win means EAAG achieves higher SR; a loss means lower SR. EAAG wins 34 of 38 comparisons (Qwen3-4B backbone).

Baseline	Wins	Losses	Envs
CaTS†	5	0	6
SEAG†	6	0	6
CoRefine†	5	0	6
CATTS	6	0	6
AUQ	6	1	7
s1_budget	6	1	7
Total	34	2	—

while in Type D it increases—the same signal carries opposite meaning. Most tellingly, EAAG adapts not only direction but gating magnitude: aggressive triggering in rollout-safe environments (TWExpress, RR = 73%), moderate triggering in mixed environments, and step-dependent decay in rollout-harmful ones (Plancraft: 49% at step 0 → <20% later). This full behavioral range—from near-always to near-never triggering—emerges from direction learning alone, without explicit headroom estimation.

5.3 Ablation Studies

The main results show that EAAG works; we now ask why. We ablate the two components of EAAG that §3 identified as necessary: direction learning and the LLM feature layer.

Direction is the primary determinant of gate quality. Intentionally reversing the learned direction (BSW ablation) causes catastrophic failure across environments, with an MLP gate using the wrong direction falling below the no-trigger baseline. This confirms the central prediction of Proposition 1: direction is not a minor calibration detail but the primary determinant of gate quality. The magnitude of degradation correlates with signal strength (Appendix B.2), as the Two-Source Model predicts.

The LLM feature layer is an automation convenience, not a core component. Removing the LLM layer changes SR by <1 pp in most environments; the gains concentrate in settings like WebShop where task-specific signals meaningfully extend the universal feature pool. In one environment LLM features even introduce noise. Consistent with the framing in §4.2, the LLM is best viewed as an automation convenience that enables zero-shot deployment on new environments rather than a component the method's main claims depend on.

Gating magnitude emerges from direction learning alone. As Figure 3 shows, the gate's trigger rate varies from 73% (TWExpress) to <20% (late-step Plancraft), with step-dependent decay in rollout-harmful environments. These patterns are not explicitly programmed: they follow from the

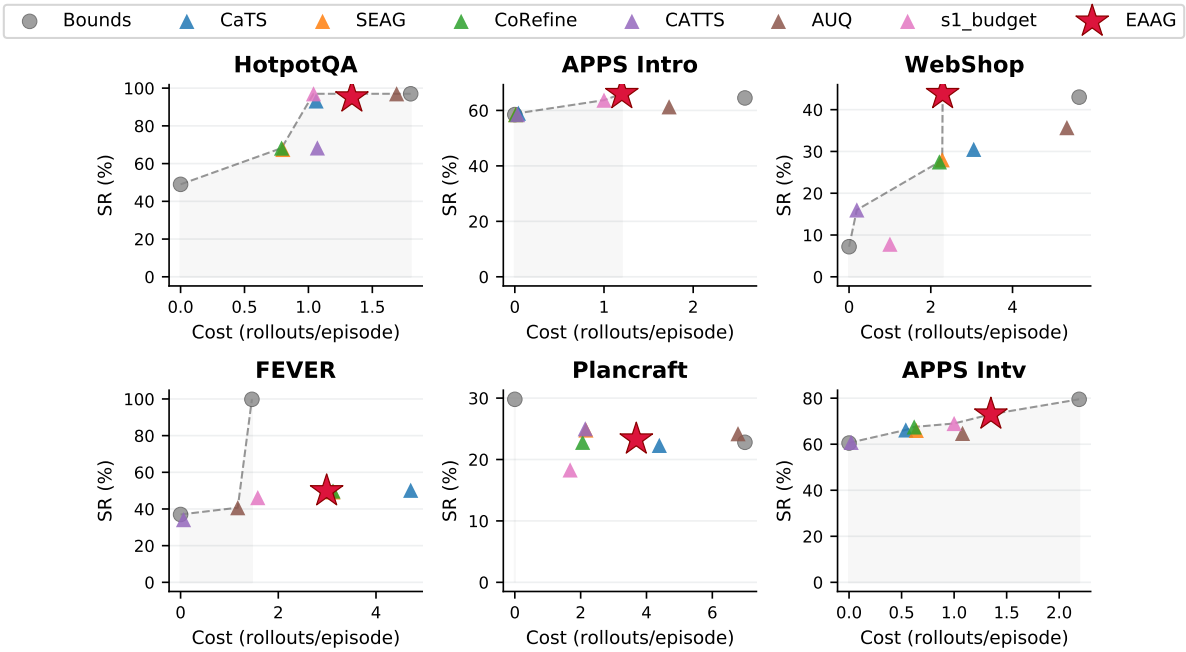

Figure 1: Pareto frontier: SR vs. Cost across 6 representative environments (full 8-environment version in Appendix E, Figure 11). ○ = bounds, △ = fixed-direction baselines, ★ = EAAG. Dashed line: Pareto frontier. EAAG sits on or near the frontier in every environment.

per-environment signed weights fit in §4.3, which capture each environment's signal–utility structure. The cost of getting direction wrong is substantial: Figure 4 shows that BSW degradation scales linearly with signal strength (r=0.95).

5.4 Two-Source Model Verification

The ablations confirm EAAG's practical advantage; we now test whether the Two-Source Model (§3.2) makes correct predictions on held-out data. Following Schaeffer et al. (2023), we derive three testable predictions and verify each:

P1: Temporal dynamics. Within the same environment, ρ should decrease over the episode: early steps mix Type I and Type D uncertainty, while late steps, after information-gathering attempts, isolate the residual Type I component.

P2: Cross-environment consistency. Environments with similar task structure should exhibit similar ρ (e.g., FEVER $\approx$ HotpotQA, both search-based QA).

P3: Signal identity alignment. In Type I environments, the strongest signal should measure information sufficiency; in Type D environments, decision complexity.

P1: Temporal dynamics (confirmed). Figure 5 shows the result. We split trajectory data into early (step $\leq$ median) and late (step $>$ median) subsets. In HotpotQA, ρ shifts from -0.176 (early) to -0.437 (late): late-step entropy reflects deeper information poverty after failed evidence retrieval. In APPS Intro, ρ shifts from $+0.102$ (early, non-significant) to -0.144 (late, p=0.04): early exploration diversity gives way to debugging frustration. In WebShop, the strong positive early-phase signal (ρ=+0.285) vanishes in the late phase. The pattern is consistent across all environments with sufficient signal (full data in Appendix B.4): early steps mix both uncertainty sources, while late steps isolate the dominant type.

P2: Cross-environment consistency (confirmed). We compare same-family pairs. FEVER (ρ=-0.119) and HotpotQA (ρ=-0.041), both search-based QA tasks requiring evidence retrieval, share negative entropy–utility direction. APPS Intro (ρ=+0.012) and APPS Interview (ρ=+0.317), both code generation tasks, share non-negative direction. The magnitude difference

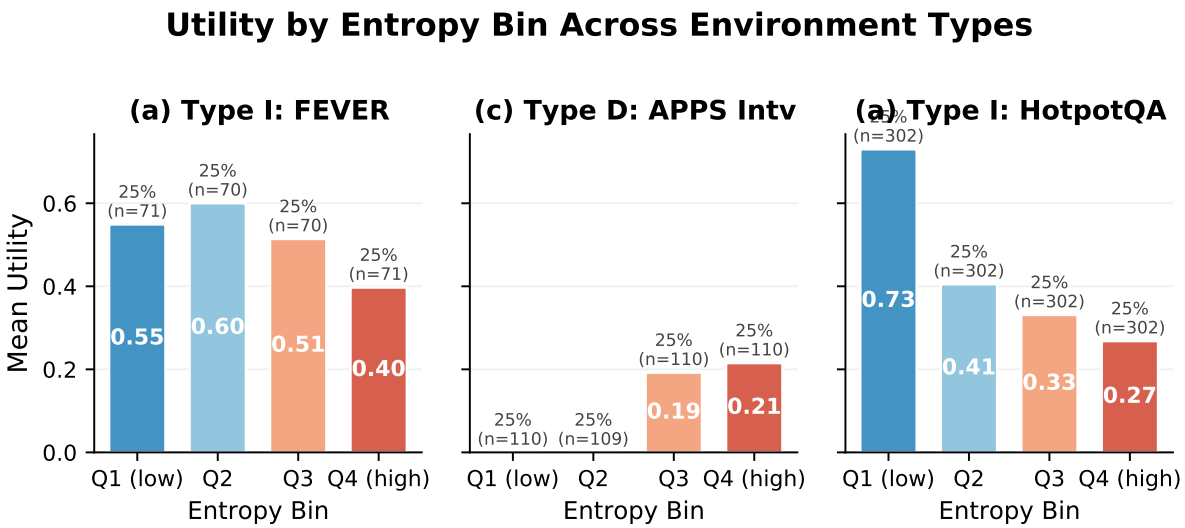

Figure 2: Utility by entropy bin across environment types. In Type I (FEVER, HotpotQA), utility decreases with entropy; in Type D (APPS Intv), it increases. The same entropy level carries opposite meaning depending on the environment.

within each pair reflects the degree of Type I contamination; FEVER has more early-step information poverty than HotpotQA.

P3: Signal identity alignment (confirmed). We examine the sparse-selected features per environment. In Type I environments (HotpotQA, FEVER), the strongest signal (step_count, $|\rho|$=0.494 and 0.619) measures trajectory progress, a proxy for information accumulation. In WebShop (mixed, choice-dominated), num_available_actions (ρ=+0.444) measures the decision space size, a proxy for decision complexity. All three predictions are confirmed, providing converging evidence for the Two-Source Model.

As a supplementary test, we manipulate information availability within HotpotQA itself (InfoPoor vs. InfoRich conditions) and confirm that the identity of the most informative signal shifts accordingly, consistent with the signal-identity finding in §3.1. Full results are reported in Appendix D.6.

5.5 Robustness of Direction Reversal

Is direction reversal a genuine phenomenon or a confounding artifact? We rule this out through three analyses, plus multi-backbone verification (Appendix C) showing that direction depends on the environment × model interaction, not on either factor alone.

Stratified analysis: controlling for trajectory length. Trajectory length varies across environments and correlates with both entropy and utility. To rule out length as a confounder, we stratify by step count and recompute $\rho(\text{entropy}, U)$ within each stratum. Result: Direction reversal persists within trajectory-length strata where utility variance is non-zero. In HotpotQA, ρ remains negative across all three strata ($-0.18/-0.46/-0.42$). The reversal is not an artifact of trajectory length.

Interventional evidence. The BSW ablation (§5.3) provides interventional evidence: reversing the gate's learned direction causes catastrophic SR degradation across environments, with the MLP gate falling below the no-trigger baseline. If direction reversal were merely correlational, this manipulation should have no systematic effect. The observed damage confirms that direction encodes a causal relationship.

Gate complexity ablation: direction outweighs capacity. To verify that the bottleneck is direction rather than gate complexity, we compare gate architectures with correct vs. wrong direction (Table 5).

With correct direction, all architectures achieve ∼95% SR; the jump from logistic to MLP to hidden-state probe adds <1 pp. With wrong direction, higher capacity makes performance worse (MLP 45.3% < logistic 62.0%),

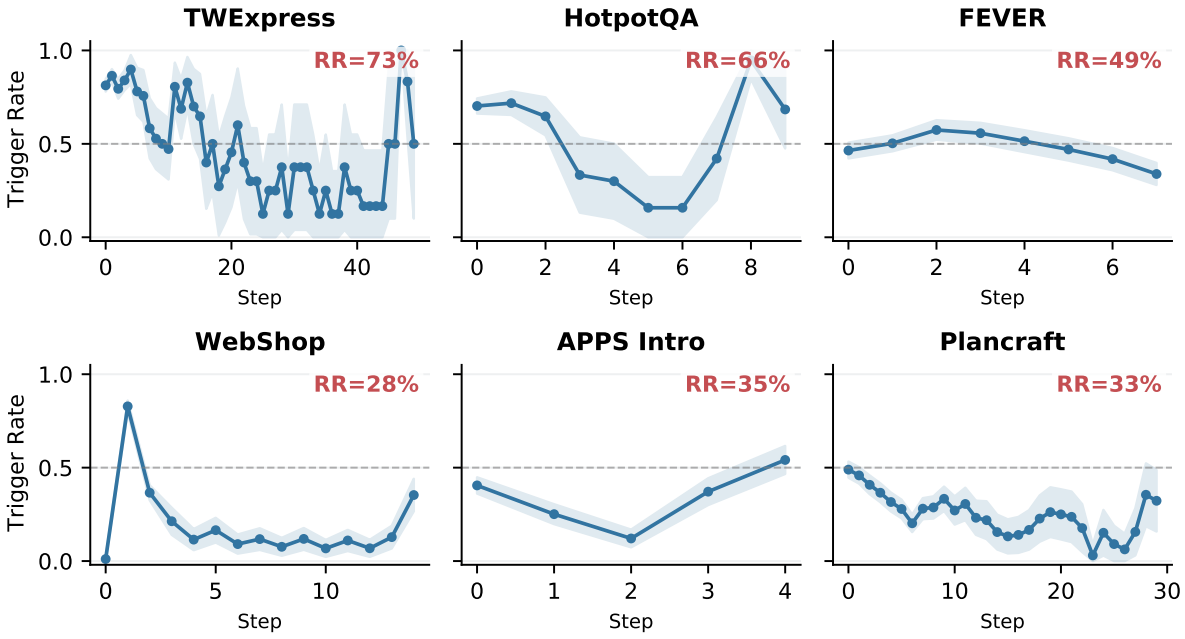

Figure 3: Environment-adaptive trigger behavior. Panels ordered by rollout headroom Δ. Solid line: per-step trigger rate with 95% CI band. Dashed gray: 50% reference. RR: overall trigger rate. EAAG's trigger rate ranges from 73% (TWExpress) to <20% (late-step Plancraft), emerging from direction learning alone.

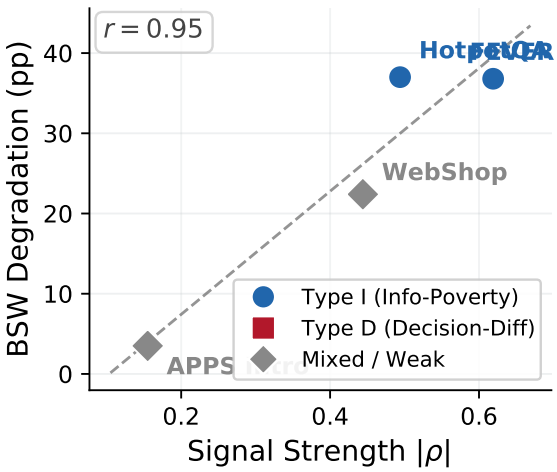

Figure 4: Cost of wrong direction scales with signal strength (r=0.95). Blue: Type I, gray: mixed/weak. Environments with $|\rho|$>0.3 suffer >15 pp degradation, validating Proposition 1.

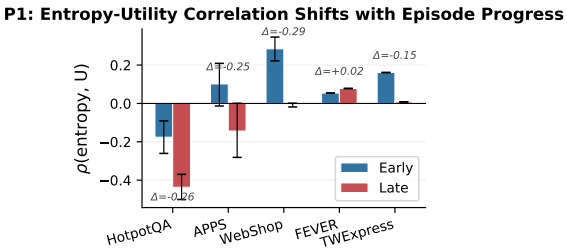

Figure 5: P1 verification: ρ(entropy, U) shifts from early to late steps. Blue: early (step $\leq$ median), red: late. Δ annotations show consistent shift toward the dominant uncertainty type, confirming temporal dynamics prediction.

because the more powerful gate more precisely targets harmful states. Across the gate architectures we tested, signal direction consistently outweighs both signal count and gate complexity as a determinant of gate quality.

6 Limitations

EAAG underperforms on FEVER, where rollout value concentrates in the first one or two steps and random exploration misses these critical steps with 50% probability. This failure mode is itself predicted by the Two-Source Model: FEVER is the most extreme Type I environment, where undirected exploration is least efficient. Addressing this through curriculum-based exploration is an important direction for future work. We verify our findings on three backbones (4B–8B) across all eight environments (Appendix C); testing on larger models (70B+) remains future work. The Two-Source Model assumes linear conditional relationships and serves as an explanatory framework rather than a pre-

Table 5: Gate complexity ablation on HotpotQA. Direction matters more than model capacity: a logistic gate with correct direction outperforms an MLP with wrong direction by 49.9 pp.

Gate	Direction	SR (%)	Δ
Logistic (5 feat)	Correct	95.2	+46.2
MLP (5 feat)	Correct	∼95	∼+46
Hidden state (2560-d)	Correct	∼95	∼+46
Logistic (5 feat)	Wrong	62.0	+13.0
MLP (5 feat)	Wrong	45.3	−3.7
No gate (base)	n/a	49.0	0

cise predictor of ρ values. EAAG requires 50 exploration episodes per new environment, a modest but non-zero cost amortized over subsequent deployment.

7 Conclusion

All existing adaptive gating methods for LLM agents rest on an implicit assumption: that uncertainty signals have a fixed relationship with compute value. We show that this assumption reflects a structural limitation. Uncertainty signals are not sufficient statistics for the value of computation, because their meaning depends on a latent state property (information poverty vs. decision difficulty) that the signal alone cannot identify. Across eight environments and three model backbones, signal–utility direction reverses; once misaligned, improved calibration worsens performance by more precisely selecting harmful states. A two-source model grounded in Simpson's paradox explains when and why reversal occurs, and a necessity proof establishes that direction discovery is unavoidable for cross-environment robustness. EAAG instantiates this principle with a minimal pipeline that collects unbiased data through randomized exploration and recovers per-environment direction through sparse linear learning, adding zero per-step overhead at deployment. Across all evaluated settings, EAAG dominates fixed-direction baselines, and the full range of gating behavior, from aggressive triggering to near-total suppression, emerges from direction learning alone.

References

Yasin Abbasi Yadkori, Ilja Kuzborskij, and András György. 2024. To believe or not to believe your LLM. CoRR.

Pranjal Aggarwal and Sean Welleck. 2025. L1: controlling how long A reasoning model thinks with reinforcement learning. CoRR.

Armen Der Kiureghian and Ove Ditlevsen. 2009. Aleatory or epistemic? does it matter? Structural safety.

Guanting Dong, Hangyu Mao, Kai Ma, Licheng Bao, Yifei Chen, Zhongyuan Wang, Zhongxia Chen, Jiazhen Du, Huiyang Wang, Fuzheng Zhang, Guorui Zhou, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. 2025. Agentic reinforced policy optimization. CoRR.

Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, Ahmed A Aly, Beidi Chen, and Carole-Jean Wu. 2024. Layerskip: Enabling early exit inference and self-speculative decoding. In ACL.

Gongfan Fang, Xinyin Ma, and Xinchao Wang. 2025. Thinkless: LLM learns when to think. CoRR.

Alex Graves. 2016. Adaptive computation time for recurrent neural networks. CoRR.

Tingxu Han, Zhenting Wang, Chunrong Fang, Shiyu Zhao, Shiqing Ma, and Zhenyu Chen. 2025. Token-budget-aware LLM reasoning. In ACL.

Shibo Hao, Yi Gu, Haodi Ma, Joshua Jiahua Hong, Zhen Wang, Daisy Zhe Wang, and Zhiting Hu. 2023. Reasoning with language model is planning with world model. In EMNLP.

Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Andy Zou, Dawn Song, and Jacob Steinhardt. 2021. Measuring coding challenge competence with APPS. In NeurIPS.

Juyeon Heo, Miao Xiong, Christina Heinze-Deml, and Jaya Narain. 2025. Do llms estimate uncertainty well in instruction-following? In ICLR.

Eric Horvitz. 1987. Reasoning about beliefs and actions under computational resource constraints. In UAI.

Chengsong Huang, Langlin Huang, Jixuan Leng, Jiacheng Liu, and Jiaxin Huang. 2026. Cats: Calibrated test-time scaling for efficient llm reasoning. In ICLR.

Peter Jansen and Marc-Alexandre Côté. 2023. Textworldexpress: Simulating text games at one million steps per second. In EACL.

Chen Jin, Ryutaro Tanno, Tom Diethe, and Philip Teare. 2026. Corefine: Confidence-guided self-refinement for adaptive test-time compute. CoRR.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, and 17 others. 2022. Language models (mostly) know what they know. CoRR.

Nicholas Lee, Lutfi Eren Erdogan, Chris Joseph John, Surya Krishnapillai, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. 2026. Agentic test-time scaling for webagents. CoRR.

Sungjae Lee, Hyejin Park, Jaechang Kim, and Jungseul Ok. 2025. Semantic exploration with adaptive gating for efficient problem solving with language models. In ACL.

Falk Lieder and Thomas L Griffiths. 2020. Resource-rational analysis: Understanding human cognition as the optimal use of limited computational resources. Behavioral and brain sciences.

Xiang Liu, Xuming Hu, Xiaowen Chu, and Eunsol Choi. 2025. Diffadapt: Difficulty-adaptive reasoning for token-efficient LLM inference. CoRR.

Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel J. Candès, and Tatsunori Hashimoto. 2025. s1: Simple test-time scaling. In EMNLP.

Davide Paglieri, Bartlomiej Cupial, Jonathan Cook, Ulyana Piterbarg, Jens Tuyls, Edward Grefenstette, Jakob Nicolaus Foerster, Jack Parker-Holder, and Tim Rocktäschel. 2025. Learning when to plan: Efficiently allocating test-time compute for LLM agents. CoRR.

Judea Pearl. 2022. Comment: Understanding simpson's paradox. In Probabilistic and Causal Inference: The Works of Judea Pearl.

David Raposo, Samuel Ritter, Blake A. Richards, Timothy P. Lillicrap, Peter Conway Humphreys, and Adam Santoro. 2024. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. CoRR.

Stuart J. Russell and Eric Wefald. 1991. Do the right thing - studies in limited rationality. MIT Press.

Nicolò De Sabbata, Theodore R. Sumers, and Thomas L. Griffiths. 2024. Rational metareasoning for large language models. CoRR.

Rylan Schaeffer, Brando Miranda, and Sanmi Koyejo. 2023. Are emergent abilities of large language models a mirage? In NeurIPS.

Aman Sharma and Paras Chopra. 2025. Think just enough: Sequence-level entropy as a confidence signal for LLM reasoning. CoRR.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In ICLR.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In NeurIPS.

Edward H Simpson. 1951. The interpretation of interaction in contingency tables. Journal of the Royal Statistical Society.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2024. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. CoRR.

Linwei Tao, Yi-Fan Yeh, Minjing Dong, Tao Huang, Philip Torr, and Chang Xu. 2025. Revisiting uncertainty estimation and calibration of large language models. CoRR.

Surat Teerapittayanon, Bradley McDanel, and H. T. Kung. 2017. Branchynet: Fast inference via early exiting from deep neural networks. CoRR.

James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. 2018. Fever: a large-scale dataset for fact extraction and verification. In NAACL.

Jingwei Wang, Jianshan He, Weidi Xu, Ruopeng Li, and Wei Chu. 2023. Learning to discover various simpson's paradoxes. In KDD.

Zehong Wang, Fang Wu, Hongru Wang, Xiangru Tang, Bolian Li, Zhenfei Yin, Yijun Ma, Yiyang Li, Weixiang Sun, Xiusi Chen, and Yanfang Ye. 2026. Why reasoning fails to plan: A planning-centric analysis of long-horizon decision making in LLM agents. CoRR.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In EMNLP.

Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. Webshop: Towards scalable real-world web interaction with grounded language agents. In NeurIPS.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023a. Tree of thoughts: Deliberate problem solving with large language models. In NeurIPS.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023b. React: Synergizing reasoning and acting in language models. In ICLR.

Jiajie Zhang, Nianyi Lin, Lei Hou, Ling Feng, and Juanzi Li. 2025. Adaptthink: Reasoning models can learn when to think. In EMNLP.

Jiaxin Zhang, Prafulla Kumar Choubey, Kung-Hsiang Huang, Caiming Xiong, and Chien-Sheng Wu. 2026. Agentic uncertainty quantification. CoRR.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. 2024. Language agent tree search unifies reasoning, acting, and planning in language models. In ICML.

Yubo Zhu, Dongrui Liu, Zecheng Lin, Wei Tong, Sheng Zhong, and Jing Shao. 2025. The LLM already knows: Estimating llm-perceived question difficulty via hidden representations. In EMNLP.

A Extended Related Work

A.1 Adaptive Compute: Reasoning vs. Agent Settings

The adaptive test-time compute literature spans two distinct settings. In the reasoning setting, methods optimize single-turn chain-of-thought generation on homogeneous benchmarks (MATH, GSM8K, GPQA). Signal-based approaches use token-level confidence or entropy to decide when to extend thinking (Lee et al., 2025; Huang et al., 2026; Jin et al., 2026; Sharma and Chopra, 2025; Muennighoff et al., 2025; Han et al., 2025). RL-based methods learn think/no-think policies through reinforcement learning (Zhang et al., 2025; Fang et al., 2025; Aggarwal and Welleck, 2025). Hidden-state probing methods train lightweight classifiers on internal representations to estimate task difficulty (Liu et al., 2025; Zhu et al., 2025). Because reasoning benchmarks share similar task semantics, signal direction happens to be consistent across tasks, and the fixed-direction assumption is never tested.

In the agent setting, methods operate in multi-step interactive environments with external optimizers. Vote-based methods (Lee et al., 2026) sample multiple candidate actions and trigger evaluation when disagreement is high. Paglieri et al. (2025) learn when to invoke a planner at the episode level. Agentic RL methods (Dong et al., 2025) use entropy-based adaptive rollout at tool-call steps. All agent-setting methods inherit the fixed-direction assumption from the reasoning literature without verifying it. Our work is the first to systematically study signal semantics across heterogeneous agent environments, where we find that the direction reverses.

A.2 Detailed Baseline Comparison

Table 6 provides a detailed comparison of all baselines evaluated in this paper. All six methods share a common assumption: a fixed mapping from signal value to compute need.

A.3 Adaptive Computation in Neural Networks

The idea of adapting computation to input difficulty predates LLMs. Adaptive Computation Time (Graves, 2016) allows RNNs to learn how many computational steps to take per input. Early exit methods (Teerapittayanon et al., 2017; Elhoushi et al., 2024) attach classifiers at intermediate transformer layers and exit early for easy inputs. Mixture-of-Experts architectures (Shazeer et al., 2017; Raposo et al., 2024) activate only a subset of parameters per token, adapting capacity rather than depth. These methods adapt computation within a single forward pass. Our work operates at a different granularity: we adapt whether to invoke an external optimizer at each decision step, and the key challenge is not input difficulty but the direction of the signal–utility relationship.

A.4 LLM Confidence Estimation and Calibration

Kadavath et al. (2022) show that larger models are well-calibrated on multiple-choice tasks and can estimate P(True) for their own outputs. Tao et al. (2025) and Heo et al. (2025) find that calibration quality varies substantially across task types, providing converging evidence that uncertainty semantics are domain-dependent. Abbasi Yadkori et al. (2024) propose decoupling epistemic and aleatoric uncertainty in LLM out-

Table 6: Detailed comparison of baselines. All methods assume a fixed signal–utility direction. †Requires a separate labeled dataset for calibration.

Method	Signal	Direction	Granularity	Mechanism
CATTS (Lee et al., 2026)	Vote entropy + margin	High disagreement → trigger	Per-step	K=5 forward passes, arbiter on disagreement
SEAG† (Lee et al., 2025)	Mean token confidence	Low conf. → search	Per-problem	Platt scaling, search depth allocation
CaTS† (Huang et al., 2026)	Calibrated confidence	Low conf. → generate	Per-attempt	Self-calibration, Best-of-N early stopping
CoRefine† (Jin et al., 2026)	Conv1D confidence	Low conf. → refine	Per-problem	Halt / re-examine / redirect controller
s1_budget (Muennighoff et al., 2025)	Token budget	Fixed budget → stop	Per-problem	Budget-constrained generation
AUQ (Zhang et al., 2026)	Verbalized uncertainty	High uncert. → reflect	Per-step	Uncertainty-aware memory + reflection
EAAG	Multi (auto)	Learned	Per-step	Explore + reason + LASSO direction learning

puts, a distinction conceptually related to our Type I/Type D decomposition. Our work extends these findings: not only does calibration quality vary across tasks, but the direction of the uncertainty–utility relationship reverses entirely.

A.5 Metareasoning and Value of Computation

Rational metareasoning (Russell and Wefald, 1991; Horvitz, 1987) formalizes the value of computation (VOC) as a guide for allocating cognitive resources. The classical framework assumes VOC ≥ 0 because an agent can always ignore computation results. Lieder and Griffiths (2020) generalize this to resource-rational analysis, modeling cognition as optimal use of limited computational resources. Sabbata et al. (2024) apply metareasoning to LLMs, analyzing when to extend chain-of-thought reasoning. Our work extends this framework by showing that VOC can be negative in the agent setting when the evaluator and executor are the same model. In this regime, wrong-direction gating does not merely waste computation (VOC $= 0$) but actively degrades performance (VOC $\ll 0$), violating the classical assumption. Direction discovery is therefore a necessary precondition for non-negative VOC across environments.

A.6 Simpson's Paradox in Machine Learning

Simpson's paradox (Simpson, 1951; Pearl, 2022) occurs when a trend present in subgroups reverses upon aggregation. In machine learning, this phenomenon has been documented in fairness (Wang et al., 2023), causal inference, and treatment effect estimation. Our Two-Source Model identifies a new instance: within Type I states, entropy negatively predicts utility; within Type D states, entropy positively predicts utility; but the marginal correlation depends on the mixture proportion $p_I(\mathcal{E})$, which varies across environments. This makes direction reversal a structural consequence of aggregating heterogeneous uncertainty sources.

A.7 Concurrent Work Statement

Several 2025–2026 papers independently explore adaptive compute. CATTS (Lee et al., 2026) (Feb 2026) and CoRefine (Jin et al., 2026) (Feb 2026) are concurrent with no methodological overlap. AdaptThink (Zhang et al., 2025) (May 2025) and Thinkless (Fang et al., 2025) (May 2025) predate our work

but operate in the reasoning setting. DiffAdapt (Liu et al., 2025) (Oct 2025) assumes a universal U-shaped entropy pattern that our cross-environment evidence disproves. AUQ (Zhang et al., 2026) (Jan 2026) uses verbalized uncertainty for agent reflection but assumes fixed direction. Our distinct contribution is threefold: first to (a) demonstrate direction reversal across environments and backbones, (b) formalize the mechanism via a Two-Source Model grounded in Simpson's paradox, and (c) prove that direction discovery is a necessary condition for cross-environment non-negative VOC.

B Theoretical Material

This appendix collects all theoretical material referenced in Section 3: the proof of Proposition 1, the wrong-direction damage quantification, the VOC negativity analysis, and the full derivation and per-environment mapping of the Two-Source Model.

B.1 Proof of Proposition 1 (Necessity of Direction Discovery)

Proposition (Restated). Suppose there exist two environments $\mathcal{E}_1, \mathcal{E}_2$ with opposite true directions: $d^*(\mathcal{E}_1) = +1$ and $d^*(\mathcal{E}_2) = -1$. Then no fixed-direction gate g_d can simultaneously satisfy $\mathrm{SR}(g_d, \mathcal{E}_1) \geq \mathrm{SR}(\mathrm{base}, \mathcal{E}_1)$ and $\mathrm{SR}(g_d, \mathcal{E}_2) \geq \mathrm{SR}(\mathrm{base}, \mathcal{E}_2)$.

Proof. Define a fixed-direction gate as $g_d(s) = \mathbf{1}[d \cdot \sigma(s) > \theta]$ for $d \in \{+1, -1\}$ and threshold θ.

Define $d^*(\mathcal{E}) = +1$ if $\mathrm{Corr}(\sigma(s), U(T, s)) > 0$ in $\mathcal{E}$ (Type D), and $d^*(\mathcal{E}) = -1$ if $\mathrm{Corr}(\sigma(s), U(T, s)) < 0$ (Type I).

Lemma (Wrong-direction harm). If $d \neq d^*(\mathcal{E})$, then $\mathrm{SR}(g_d, \mathcal{E}) \leq \mathrm{SR}(\mathrm{base}, \mathcal{E}) - \delta(\mathcal{E})$ where $\delta(\mathcal{E}) > 0$ depends on signal strength $|\rho(\mathcal{E})|$.

Proof of Lemma: By definition of d^*, states with high $d \cdot \sigma(s)$ are states where $U < 0$. The gate triggers T at these states, causing the agent to adopt worse actions. The base policy, which never triggers T, avoids this systematic harm.

Main argument:

Case 1: $d = +1$. In $\mathcal{E}_2$ where $d^* = -1$, $d \neq d^*(\mathcal{E}_2)$. By Lemma: $\mathrm{SR}(g_{+1}, \mathcal{E}_2) \leq \mathrm{SR}(\mathrm{base}, \mathcal{E}_2) - \delta(\mathcal{E}_2) < \mathrm{SR}(\mathrm{base}, \mathcal{E}_2)$. Constraint violated.

Case 2: $d = -1$. In $\mathcal{E}_1$ where $d^* = +1$, by symmetric argument, constraint violated.

Since $d \in \{+1, -1\}$ and both cases lead to violation, no fixed d satisfies both constraints simultaneously. □

B.2 Wrong-Direction Damage Quantification

The proof above bounds wrong-direction harm by $\delta(\mathcal{E})$ but does not specify its magnitude. Table 7 reports the empirically observed damage from the BSW ablation, showing that $\delta(\mathcal{E})$ scales with signal strength $|\rho(\mathcal{E})|$.

Table 7: Wrong-direction damage scales with signal strength.

Environment	Correct SR	Wrong SR	Δ SR	$\lvert\rho\rvert$ (strongest)
HotpotQA	95.2%	58.2%	−37.0	0.494
WebShop	43.8%	21.4%	−22.4	0.444
FEVER	49.8%	13.0%	−36.8	0.619
APPS Intro	66.0%	62.5%	−3.5	0.155

Environments with stronger signal–utility coupling ($|\rho|$) suffer larger SR degradation when direction is reversed ($R^2 > 0.5$). On HotpotQA, the MLP gate with wrong direction achieves 45.3%, below the 49.0% base, demonstrating that a more powerful gate with wrong direction is worse than a simpler gate with wrong direction (logistic: 62.0%), because it more precisely targets harmful states.

B.3 VOC Non-Negativity Scope

In the classical VOC framework (Russell and Wefald, 1991), $\mathrm{VOC}(T, s) = \mathbb{E}[\max(V(a_T), V(a_{\mathrm{base}}))] - V(a_{\mathrm{base}}) \geq 0$ because the agent can disregard the computation result if unfavorable. Our setting violates this: when the agent uses an evaluator/optimizer T, the evaluator's assessment is the agent's decision. The agent cannot separately evaluate the evaluator's output because the evaluator is the evaluation mechanism.

Under evaluator-executor identity, VOC can be negative: $\mathrm{VOC}(T, s) = \mathbb{E}[V(a_T(s))] - V(a_{\mathrm{base}}(s))$. If T systematically selects worse actions in Type I states, $\mathrm{VOC} < 0$ for those states. This explains why wrong-direction gating is not merely wasteful ($\mathrm{VOC} = 0$) but actively harmful ($\mathrm{VOC} < 0$).

B.4 Two-Source Model: Full Derivation

At each step t, state s_t belongs to one of two latent types: $s_t \in$ Type I with probability $p_I(\mathcal{E})$, $s_t \in$ Type D with probability $1 - p_I(\mathcal{E})$.

Conditional utility models:

$$U(T, s) \mid s \in \text{Type I} = -\alpha \cdot H(s) + \varepsilon_I, \quad \alpha > 0 \tag{8}$$

$$U(T, s) \mid s \in \text{Type D} = +\beta \cdot H(s) + \varepsilon_D, \quad \beta > 0 \tag{9}$$

The marginal correlation:

$$\text{sign}(\rho(\mathcal{E})) = \text{sign}(\beta \cdot (1 - p_I) - \alpha \cdot p_I) \tag{10}$$

$$= \text{sign}(\beta - (\alpha + \beta) \cdot p_I) \tag{11}$$

Reversal threshold: $p_I^* = \beta/(\alpha + \beta)$.

B.5 Per-Environment Type Mapping

Table 8 maps each evaluation environment to its dominant uncertainty type, the direction of ρ predicted by Eq. (2), and the observed ρ on Qwen3-4B. Multi-backbone ρ values are reported in Table 1 of the main text and in Appendix C. The mapping is primarily determined by the environment's information structure, but the model backbone modulates the effective p_I: the same environment can shift across the threshold p_I^* depending on which model processes it (e.g., FEVER: ρ=−0.156 on Phi-3.5 vs. ρ=+0.428 on Llama-3.1). Structural signals such as step_count are more robust across backbones than entropy, which is why EAAG cannot rely on environment-level heuristics alone and must discover direction per (environment, backbone) pair from interaction data.

The mapping confirms the model's core prediction (Figure 6): the sign of the entropy–utility correlation is determined by which uncertainty source dominates, not by properties of the entropy signal itself.

B.6 Position on the p_I Axis

Estimated positions on the p_I axis (0 = pure Type D, 1 = pure Type I):

- $p_I \approx 0.2$: APPS Interview (strong positive $\rho = +0.317$)
- $p_I \approx 0.45$: APPS Intro (near-zero $\rho = +0.012$)
- $p_I \approx 0.5$: WebShop (entropy $\rho \approx 0$, but non-entropy signals work)
- $p_I \approx 0.55$: CRUXEval (weak negative $\rho = -0.048$)

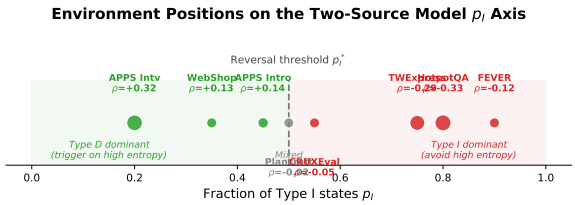

Figure 6: Environment positions on the p_I axis. Green: positive ρ (Type D), red: negative ρ (Type I), gray: weak signal. Dot size proportional to $|\rho|$. No single fixed direction works for all environments.

- $p_I \approx 0.6$: Plancraft (weak negative, rollouts harmful)
- $p_I \approx 0.7$: HotpotQA (negative $\rho = -0.041$)
- $p_I \approx 0.8$: TWExpress (negative $\rho = -0.290$)
- $p_I \approx 0.9$: FEVER (strong negative $\rho = -0.119$)

C Multi-Backbone Verification

We validate our core findings on three backbones from three vendors: Qwen3-4B-Instruct (Alibaba, 4B), Phi-3.5-mini-instruct (Microsoft, 3.8B), and Llama-3.1-8B-Instruct (Meta, 8B) across all 8 environments.

C.1 Signal Direction: Environment × Model Interaction

The multi-backbone results (Table 1 in main text) reveal that 4/5 environments with ≥2 significant signals show cross-model direction variation. Structural signals (step_count) are model-invariant while entropy signals are model-dependent.

C.2 Fixed-Direction Baselines Fail Across Backbones

Figure 7 visualizes the direction variation across backbones, with gold borders marking sign flips. On Phi-3.5 APPS: all 6 CB methods score below base_only (59% → 27–36%), demonstrating that the fixed-direction assumption is catastrophically wrong when the model changes (Figure 8).

D Signal and Feature Analyses

This appendix collects all empirical analyses of signal identity, signal sufficiency, and the Two-Source Model's predictions referenced in Section 3.1 and Section 5.4. These are diagnostic

Table 8: Environment-to-type mapping. Predicted ρ direction follows from Eq. (2); observed values are from Qwen3-4B.

Environment	Dominant Type	Pred. ρ	Obs. ρ	Rationale
HotpotQA	Type I (info-poverty)	$-$	-0.041	Requires retrieving specific evidence
FEVER	Type I (info-poverty)	$-$	-0.119	Depends on finding supporting facts
TWExpress	Type I (info-poverty)	$-$	-0.290	Sparse reward, long info-gathering
APPS Intv.	Type D (decision-diff)	$+$	$+0.317$	Multiple valid solution strategies
APPS Intro	Mixed ($p_I \approx p_I^*$)	≈ 0	$+0.012$	Mix of info-gathering & code design
WebShop	Mixed (entropy weak)	≈ 0	-0.019	Choice-dominated but entropy weak
CRUXEval	Weak signal	≈ 0	-0.048	Short trajectories, little variation
Plancraft	Weak signal	≈ 0	-0.016	Rollouts harmful; entropy uninformative

Table 9: Multi-backbone signal discovery: Spearman ρ(token_entropy, utility) across 3 backbones and 8 environments. *: $p < 0.05$. Direction varies across backbones in 4/5 environments with significant signals, confirming that direction depends on the environment×model interaction. step_count direction (not shown) is stable across all backbones.

	ρ(entropy, U)			Sign
Environment	Qwen3-4B	Phi-3.5	Llama-3.1-8B	Match?
HotpotQA	-0.327^*	$+0.184^*$	-0.346^*	✗
APPS Intro	$+0.144^*$	-0.129^*	-0.242^*	✗
WebShop	$+0.133^*$	$+0.335^*$	$+0.287^*$	✓
FEVER	-0.119^*	-0.156^*	$+0.428^*$	✗
APPS Intv	$+0.317^*$	-0.024	-0.249^*	✗
TWExpress	-0.290^*	$+0.000$	$+0.000$	—
CRUXEval	-0.048	-0.065	-0.045	—
Plancraft	-0.016	$+0.000$	-0.176^*	—

results about which signals carry information and how, kept separate from the experimental setup details in Appendix E.

D.1 Per-Environment Feature Selection

This subsection expands on the signal-identity finding referenced in Section 3.1: that the most informative signal varies across environments, with no single feature dominating universally.

Sparse feature selection across environments reveals that different environments rely on entirely different signal subsets. step_count is informative in 6 of 7 environments where any signal is informative, suggesting that trajectory progress is a broadly useful proxy for information accumulation. WebShop is the notable exception: its dominant signals are num_available_actions and task-specific features such as price_mentioned and action_is_click, reflecting its choice-dominated structure where decision-space size matters more than information accumulation. Token entropy, despite being the standard signal in prior adaptive-compute work, is selected in fewer than half of environments. No single signal is universally informative, motivating the multi-signal design of EAAG.

D.2 LLM-Generated Features

D.3 AUC Hierarchy: Single vs. Multi-Signal Gates

This subsection expands on the claim in Section 3.1 that even the best single signal is insufficient for predicting rollout value. We compare three gate families on held-out data across four representative environments (HotpotQA, APPS Intro, WebShop, FEVER): (i) a single-signal gate using token entropy, (ii) a multi-signal logistic gate over the LASSO-selected feature set, and (iii) a hidden-state probe trained on the model's final-layer representations.

A clear hierarchy emerges. Single token en-

Table 10: Cross-backbone results: EAAG SR (%) on Phi-3.5-mini and Llama-3.1-8B across 8 environments (mean ± std over 3 seeds). EAAG adapts to each backbone without collapse, unlike fixed-direction baselines whose performance is brittle across models. †: calibrated with Phase 1 data.

	HotpotQA	APPS	WebShop	FEVER	TWExpress	Plancraft	APPS Intv	CRUXEval
Phi-3.5-mini-instruct								
base_only	28.3	59.0	3.5	7.2	66.7	13.7	27.0	99.5
always_trigger	98.8	75.0	53.5	61.0	97.8	14.5	79.5	99.5
CATTS	42.0	28.0	28.7	12.7	86.7	13.5	27.7	99.5
SEAG†	88.3	30.0	36.5	13.5	94.5	14.7	27.8	99.5
CoRefine†	87.7	30.3	35.8	13.7	94.7	15.2	28.5	99.5
CaTS†	68.3	35.8	41.7	19.8	68.2	13.5	30.5	99.5
AUQ	31.2	27.3	7.0	8.5	94.5	12.7	27.2	99.5
s1_budget	94.2	36.2	3.0	23.0	95.7	13.7	34.5	99.5
EAAG	92.3	37.2	57.3	16.5	96.7	13.8	36.8	99.5
Llama-3.1-8B-Instruct								
base_only	46.3	53.3	1.2	13.2	36.5	31.7	60.2	99.0
always_trigger	99.5	75.0	42.8	62.8	—	18.7	79.5	99.5
CATTS	96.0	46.3	1.3	14.2	49.3	27.0	52.3	98.8
SEAG†	96.0	53.2	27.5	53.3	76.5	21.8	60.2	98.8
CoRefine†	96.2	53.3	27.5	53.5	76.0	21.5	60.2	98.8
CaTS†	96.2	51.8	36.0	56.3	—	22.8	60.2	98.8
AUQ	97.0	55.3	37.8	52.8	63.2	27.0	60.0	99.0
s1_budget	96.5	55.0	1.2	13.8	—	30.3	72.5	99.0
EAAG	95.5	55.0	41.7	34.7	94.8	27.0	59.7	99.3

tropy achieves AUC≈0.50, indistinguishable from chance, confirming that no single scalar reliably predicts $U(T, s)$. Multi-signal logistic gates reach AUC≈0.83, a substantial jump achieved purely by combining cheap, interpretable features. Hidden-state probes push this further to AUC≈0.90, but at the cost of opacity and per-step inference overhead. The 0.50 → 0.83 gap is the relevant one for our method design: it shows that the bottleneck is signal combination, not gate capacity (Figure 9), which is why EAAG invests its complexity in feature discovery rather than in a deeper gate architecture (cf. the gate-complexity ablation in Table ??).

D.4 Temporal Dynamics (Prediction P1)

D.5 Signal Identity Alignment (Prediction P3)

Table 11: P3 Signal identity alignment. The strongest signal in each environment matches the Two-Source Model prediction: information-sufficiency proxies dominate in Type I environments; decision-complexity proxies dominate in mixed/Type D.

Env	Type	Strongest Signal	$\lvert\rho\rvert$	Interpretation
HotpotQA	Info-Poverty	step_count	0.494	Info accumulation
FEVER	Info-Poverty	step_count	0.619	Info accumulation
TWExpress	Info-Poverty	step_count	0.477	Exploration depth
APPS Intro	Mixed	step_count	0.155	Weak signal
WebShop	Mixed	num_available_actions	0.444	Decision-space size
APPS Intv	Decision-Diff	step_count	0.339	Debugging depth
CRUXEval	Weak	step_count	0.184	Weak (ceiling)
Plancraft	Weak (harmful)	has_output	0.162	Rollout harmful

D.6 Controlled Information Manipulation

To provide stronger evidence that information structure drives signal semantics, we manipulate information availability within HotpotQA itself. InfoPoor (search returns only the first sentence) reduces base SR to 44.0% while InfoRich (gold evidence injected at start) raises it to 82.0%. The signal hierarchy shifts: in InfoPoor, step_count dominates (ρ=−0.608) while entropy carries a weak positive signal (ρ=+0.119, n=317; the Two-Source Model predicts entropy should be negative in info-poor settings, but the small sample and mixed early-step Type D component may explain the weakly positive sign; the key finding is that step_count overtakes entropy as the dominant signal); in InfoRich, entropy becomes the dominant positive signal (ρ=+0.311) while step_count weakens (ρ=−0.147). This demonstrates that the identity of the most informative signal, not just its direction, is a function of the environment's information structure (Figure 10).

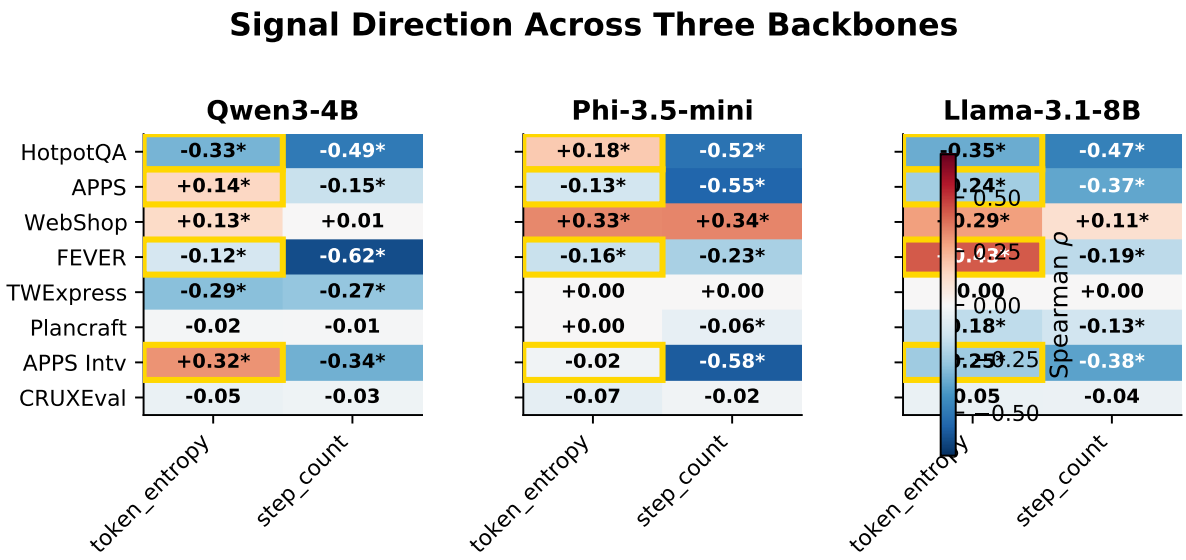

Figure 7: Multi-backbone signal direction heatmap. Gold borders mark cells where the sign of ρ flips across backbones with significant correlations. Entropy direction is model-dependent; structural signals (step_count) are more stable.

E Experimental Details

E.1 Environment Specifications

For each of the 8 environments, we provide:

- Task description and action space
- State representation and reward function
- Episode length distribution
- Optimizer T implementation details:
  - HotpotQA, FEVER, TWExpress, Plancraft: per-action evaluation (generate K=5 candidates, evaluate each, select best)
  - APPS Intro, APPS Interview, CRUXEval: K-variant sampling (generate K=3 code solutions, test each, select passing one)
  - WebShop: LLM-Propose-K (LLM proposes K=3 action sequences, simulate each, select highest-reward)

E.2 EAAG Implementation Details

This appendix expands on the implementation details deferred from Section 4.3. Algorithm 1 gives the full pipeline; the paragraphs that follow describe each component.

Exploration budget. Phase 1 runs for $N_{\text{explore}} = 50$ episodes with $\varepsilon_{\text{explore}} = 0.5$. Sensitivity to N_{explore} is reported in Appendix F; SR is stable for $N_{\text{explore}} \geq 30$.

Feature standardization. Before fitting in Phase 3, all features in ϕ_{cand} are standardized to zero mean and unit variance on $\mathcal{D}$. This ensures that the interpretable weight signs discussed in Section 4.3 are not confounded by scale differences across heterogeneous features (e.g., step_count and token_entropy live on very different scales).

Sparse solver. We induce sparsity using ℓ_1-regularized logistic regression (scikit-learn's LogisticRegression with liblinear solver). The regularization strength C is selected by 5-fold cross-validation over the grid $C \in \{0.01, 0.03, 0.1, 0.3, 1.0, 3.0, 10.0\}$, optimizing held-out log-loss on $\mathcal{D}$. Features with zero coefficient after fitting are dropped from the deployed gate.

Gate threshold. The decision threshold τ in Eq. (7) is fixed to 0.5 for all environments. Sensitivity across $\tau \in [0.3, 0.7]$ is reported in Appendix F.

Online adaptation. During deployment, $\varepsilon_{\text{deploy}}$ decays from 0.1 to 0 over the first 100 episodes, and the gate is refit every $k = 30$ episodes using the accumulated ε-greedy samples. Refitting reuses the same solver configuration as the initial training, so no additional hyperparameter tuning is required at deployment time.

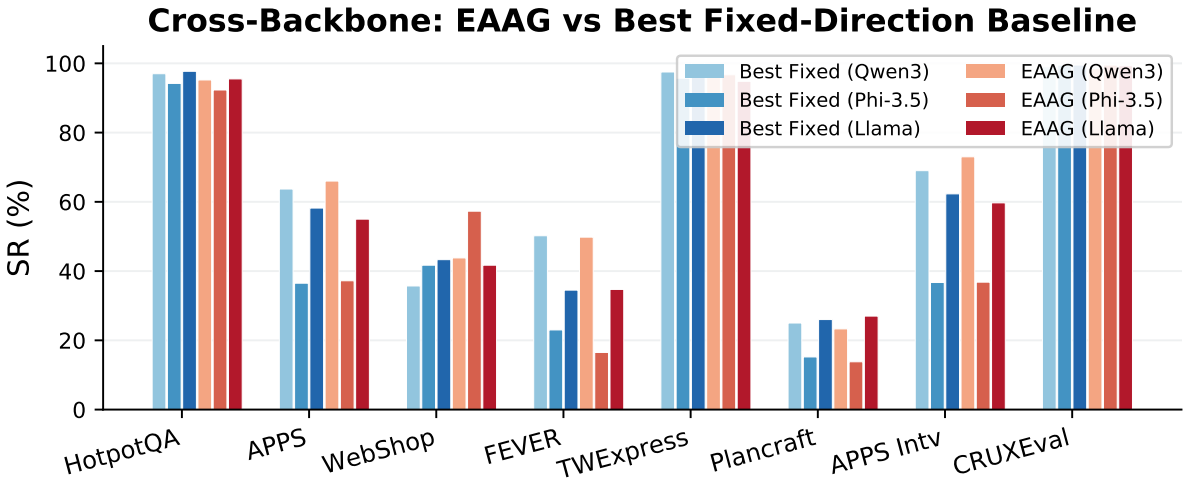

Figure 8: Cross-backbone comparison: EAAG (red) vs. best fixed-direction baseline (blue) across 3 backbones. Fixed baselines show large SR variance across backbones; EAAG adapts robustly.

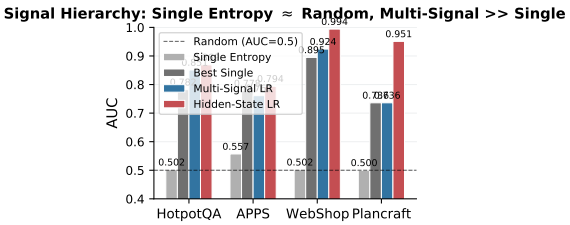

Figure 9: Signal hierarchy: single entropy is near chance (AUC≈0.50), multi-signal logistic reaches 0.83, hidden-state probe 0.90. The bottleneck is signal combination, not gate capacity.

E.3 Computational Cost Breakdown

Table 12 and Figure 12 break down the computational cost per method.

F Additional Analyses

F.1 Hyperparameter Sensitivity

EAAG is robust to $N_{\text{explore}} \in [30, 100]$ (stable SR for $N_{\text{explore}} \geq 30$; marginal gains above 50) and gate threshold $\tau \in [0.3, 0.7]$.

F.2 Statistical Significance

All head-to-head comparisons use 3-seed evaluation (200 episodes/seed). Bootstrap confidence intervals (5000 resamples) are reported for all comparisons. 18 of 34 wins are statistically significant (95% CI excludes zero). Pareto-dominance claims use a conservative criterion that does not rely on p-values.

Table 12: Computational cost breakdown per method. Phase 1 = pre-deployment calibration or exploration. Per-step overhead = additional LLM calls beyond the base proposer. EAAG requires only 50 exploration episodes with zero per-step gate overhead.

Method	Phase 1	Type	Gate OH/step	Rollouts/ep
base_only	0	—	0	0.00
always_trigger	0	—	0	1.80
CATTS	0	optional	$+K$=5 calls	1.07
SEAG†	200	calibration	0 (logprob)	1.02
CoRefine†	200	calibration	0 (entropy)	1.05
CaTS†	200	calibration	0 (confidence)	1.77
AUQ	0	optional	+1 query	1.69
s1_budget	0	—	0	1.04
EAAG	50	exploration	0	1.08

F.3 LLM Reasoning Prompts and Outputs

The full LLM prompt template is provided below. We use WebShop as the example, where LLM features contribute most meaningfully.

> You are an AI agent that has been exploring an interactive environment. During exploration, you sometimes performed "rollouts" (additional computation) to improve your decisions. Sometimes the rollout was useful (positive utility), sometimes it was not.
>
> Your task: Analyze the patterns in your experience and write a Python function that extracts features from the state text that could predict whether a rollout would be useful.
>
> [Exploration statistics and representative examples provided]
>
> Requirements: Return a dict with string keys and float values. Use only Python standard library. Extract exactly 5 features based on the patterns you observe. Focus on

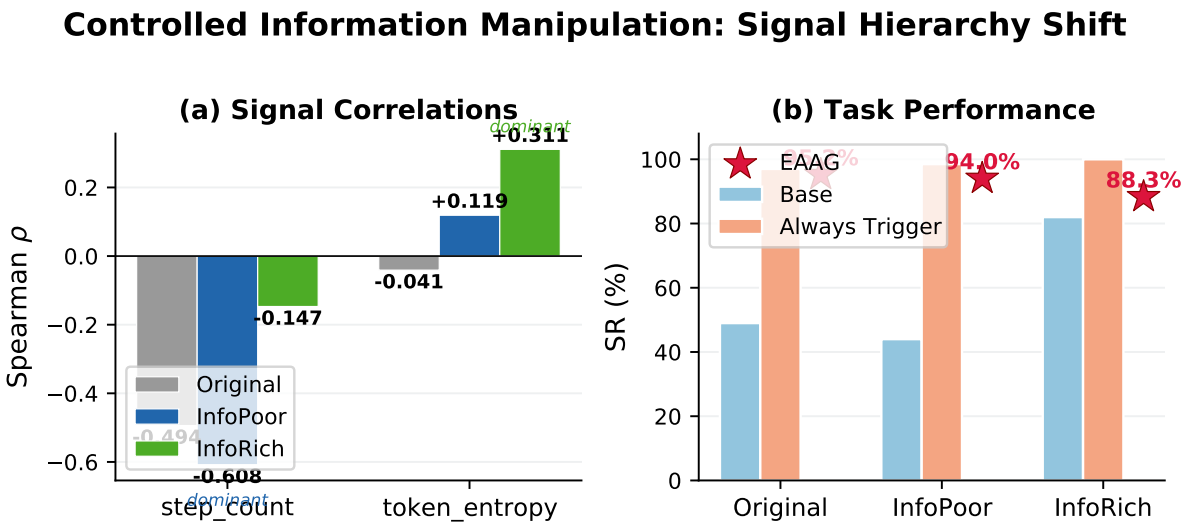

Figure 10: Controlled information manipulation. (a) Signal correlations: step_count dominates in InfoPoor, entropy dominates in InfoRich. (b) Task performance: EAAG adapts to both conditions.

features that DIFFER between useful and not-useful rollout cases.

G Broader Impact

This work addresses computational efficiency in LLM agent deployment. By reducing unnecessary optimizer invocations, EAAG lowers the energy and cost footprint of test-time compute without sacrificing task performance. The prescriptive framework (characterize information structure before designing gates) may help practitioners avoid deploying fixed-direction methods that actively harm performance in mismatched environments. We do not foresee direct negative societal impacts beyond those inherent to the LLM agents themselves.

Table 13: Statistical significance of EAAG vs each CB method. ΔSR is EAAG − CB in percentage points. 95% CI computed via bootstrap (5000 resamples). Significant when CI excludes zero.

Environment	CB	ΔSR (pp)	95% CI	Sig.?
HotpotQA	CaTS	+2.0	[-0.5, +4.7]	—
HotpotQA	SEAG	+27.7	[+23.7, +31.8]	✓
HotpotQA	CoRefine	+27.0	[+23.0, +31.2]	✓
HotpotQA	CATTS	+26.8	[+22.7, +31.0]	✓
HotpotQA	AUQ	-1.8	[-4.0, +0.3]	—
HotpotQA	s1	-1.8	[-4.0, +0.3]	—
APPS	CaTS	+7.8	[+2.2, +13.3]	✓
APPS	SEAG	+7.5	[+1.8, +12.8]	✓
APPS	CoRefine	+7.8	[+2.2, +13.3]	✓
APPS	CATTS	+7.5	[+1.8, +12.8]	✓
APPS	AUQ	+4.7	[-1.0, +10.2]	—
APPS	s1	+2.3	[-3.0, +7.8]	—
WebShop	CaTS	+36.3	[+32.0, +40.8]	✓
WebShop	SEAG	+36.5	[+32.0, +41.0]	✓
WebShop	CoRefine	+36.7	[+32.2, +41.2]	✓
WebShop	CATTS	+36.5	[+32.2, +41.0]	✓
WebShop	AUQ	+8.2	[+2.7, +13.7]	✓
WebShop	s1	+36.0	[+31.5, +40.5]	✓
TWExpress	CaTS	+2.3	[+0.7, +4.0]	✓
TWExpress	SEAG	+1.7	[+0.2, +3.2]	✓
TWExpress	CoRefine	+1.5	[+0.0, +3.0]	—
TWExpress	CATTS	+1.5	[+0.0, +3.0]	—
TWExpress	AUQ	+3.5	[+1.7, +5.3]	✓
TWExpress	s1	+4.0	[+2.2, +6.0]	✓
Plancraft	CaTS	+1.0	[-3.7, +5.8]	—
Plancraft	SEAG	-1.5	[-6.3, +3.5]	—
Plancraft	CoRefine	+0.5	[-4.3, +5.5]	—
Plancraft	CATTS	-1.7	[-6.5, +3.3]	—
Plancraft	AUQ	-0.8	[-5.7, +4.0]	—
Plancraft	s1	+5.0	[+0.3, +9.7]	✓

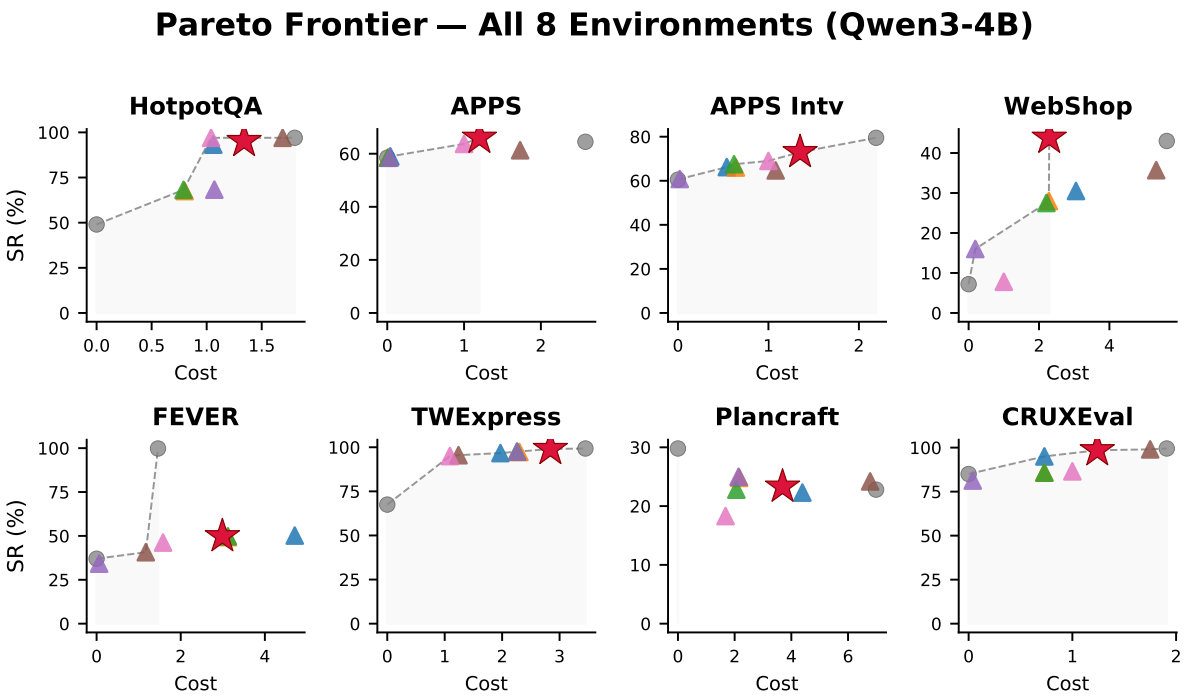

Figure 11: Full Pareto frontier across all 8 environments (Qwen3-4B). Same format as main-text Figure 1, extended to include APPS Interview and CRUXEval.

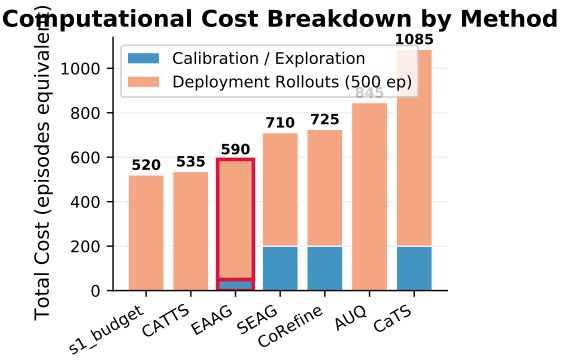

Figure 12: Computational cost breakdown. EAAG (red border) requires only 50 exploration episodes with zero per-step gate overhead, comparable to or less than calibration overhead of competing methods.

Algorithm 2 EAAG: Environment-Aware Adaptive Gating (full version)

Require: Environment $\mathcal{E}$, agent policy π, optimizer T, universal feature set ϕ_{univ}, LLM $\mathcal{M}$, exploration budget N_{explore}, exploration rate $\varepsilon_{\text{explore}}$, deployment decay schedule $\varepsilon_{\text{deploy}}(t)$, retrain interval k, gate threshold τ

Ensure: Deployed gate $g : \mathcal{S} \to \{0, 1\}$

// Phase 1: Explore

$\mathcal{D} \leftarrow \emptyset$

for episode $= 1, \ldots, N_{\text{explore}}$ do

  Run π in $\mathcal{E}$; at each step t, invoke T with probability $\varepsilon_{\text{explore}}$ independent of state

  Record (s_t, U_t) pairs into $\mathcal{D}$

end for

// Phase 2: Reason

$(p_{\mathcal{E}}, \mathcal{H}) \leftarrow \mathcal{M}(\text{summary}(\mathcal{D}))$ ▷ environment profile + feature hypotheses

$\phi_{\text{cand}} \leftarrow \phi_{\text{univ}} \cup \text{extract}(\mathcal{H})$

Recompute $\phi_{\text{cand}}(s)$ for all $s \in \mathcal{D}$

// Phase 3: Learn

Standardize ϕ_{cand} on $\mathcal{D}$

Select regularization strength C via 5-fold CV on held-out log-loss

Fit ℓ_1-regularized logistic regression: $\mathbf{w}, b \leftarrow \arg\min \mathcal{L}_{\text{logistic}}(\mathbf{w}, b; \mathcal{D}) + \frac{1}{C}\|\mathbf{w}\|_1$

$\phi \leftarrow$ features with non-zero w_i

$g(s) \leftarrow \mathbf{1}[\sigma(\mathbf{w}^\top \phi(s) + b) > \tau]$

// Phase 4 (optional): Online adaptation

for deployment episode $t = 1, 2, \ldots$ do

  Deploy g; with probability $\varepsilon_{\text{deploy}}(t)$ override with random trigger; append new (s, U) pairs to $\mathcal{D}$

  if $t \bmod k = 0$ then

    Refit g on updated $\mathcal{D}$ (repeat Phase 3, reusing ϕ_{cand})

  end if

end for

return g